

To generate a random array with a parameterized ratio of adjacent elements being "out of order," you can follow these steps:

Algorithm Explanation

1. **Generate a Sorted Array:** Start with a sorted array of size n .
2. **Determine the Number of Swaps:** Calculate the number of adjacent swaps needed based on the desired ratio r (e.g., if $r = 0.5$, 50% of adjacent elements will be out of order).
3. **Perform Random Swaps:** Randomly select adjacent pairs and swap them until the required number of swaps is achieved.

Python Implementation

Here's an example implementation:

```
import random

def generate_random_array(n, ratio):
    # Step 1: Create a sorted array
    array = list(range(1, n + 1))

    # Step 2: Calculate the number of swaps
    num_swaps = int(ratio * (n - 1)) # (n-1) is the total number of adjacent
    pairs

    # Step 3: Perform random adjacent swaps
    for _ in range(num_swaps):
        # Randomly pick an index for swapping adjacent elements
        i = random.randint(0, n - 2) # Ensure i and i+1 are valid indices
        array[i], array[i + 1] = array[i + 1], array[i]

    return array

# Example usage
n = 10 # Size of the array
```

```
ratio = 0.4 # 40% of adjacent elements out of order
random_array = generate_random_array(n, ratio)
print(random_array)
```

Key Points

1. Input Parameters:

1. **n**: Size of the array.
2. **ratio**: Fraction of adjacent elements to be out of order ($0 \leq \text{ratio} \leq 1$).

2. **Output**: A random array with the specified ratio of adjacent elements out of order.

3. **Efficiency**: The algorithm runs in $O(n)$ time for generating the array and performing swaps.

This approach ensures flexibility and randomness while adhering to the specified ratio. Let me know if you'd like further clarification!